

LAPORAN PRAKTIKUM BIG DATA
PRAKTIKUM 2
PENGUMPULAN DAN PRA-PEMROSESAN DATA
(DATA ACQUISITION & PREPROCESSING)



Disusun oleh:

Habibi Putra Rizqullah

NIM 2411531001

Kelas A

Dosen Pengampu:

Afdhal Dinilhak, M.Kom, S.Kom

FAKULTAS TEKNOLOGI INFORMASI
PROGRAM STUDI INFORMATIKA
UNIVERSITAS ANDALAS
2026

DAFTAR ISI

BAB I PENDAHULUAN	2
1.1 Latar Belakang	2
1.2 Tujuan Praktikum	2
BAB II DASAR TEORI	3
2.1 Mengapa Data Mentah Perlu Dibersihkan	3
2.2 Kaitan dengan Karakteristik 5V	3
2.3 Posisi Praktikum dalam Pipeline Big Data	3
2.4 Missing Value dan Strategi Penanganannya	3
2.5 Duplicate Data	4
2.6 Koreksi Tipe Data dan Standardisasi Format	4
2.7 Dataset Sintetis dan Faker	4
BAB III METODOLOGI PRAKTIKUM	5
3.1 Waktu dan Lingkungan Praktikum	5
3.2 Perangkat dan Perangkat Lunak	5
3.3 Dataset	5
3.4 Akses Notebook dan Repository GitHub	6
3.5 Tahapan Praktikum	6
BAB IV HASIL DAN PEMBAHASAN	7
4.1 Import Library dan Inisialisasi (K-1)	7
4.2 Membuat Dataset Sintetis (K-2)	7
4.3 Deteksi dan Penanganan Missing Value (K-3)	9
4.4 Deteksi dan Penanganan Duplicate (K-4)	10
4.5 Koreksi Tipe Data dan Standardisasi Format (K-5)	10
4.6 Ekspor Dataset Bersih (K-6)	13
4.7 Analisis Hasil	13
4.8 Studi Kasus	14
4.9 Latihan Praktikum	15
4.9.1 Perbandingan Pipeline SEED 42 dan SEED 7	15
4.9.2 Kolom is_valid_price	17
4.9.3 Jumlah Transaksi per Kategori	17
BAB V TUGAS PRAKTIKUM DAN PERTANYAAN EVALUASI	18
5.1 Tugas Praktikum	18
5.2 Pertanyaan Evaluasi	18
BAB VI PENUTUP	20
6.1 Kesimpulan	20
DAFTAR PUSTAKA	21
LAMPIRAN	22

BAB I

PENDAHULUAN

1.1 Latar Belakang

Data yang berasal dari sumber transaksional seperti sistem “point of sale”, aplikasi pembayaran, atau platform marketplace, direkam oleh banyak sistem yang berbeda, diinput oleh banyak pengguna, dan pada waktu yang berbeda-beda. Kondisi ini membuat data mentah yang diterima hampir selalu memiliki masalah kualitas, mulai dari baris yang datanya kosong (missing value), transaksi yang tercatat dua kali (duplicate), hingga kolom yang formatnya tidak konsisten satu sama lain.

Praktikum ini merupakan kelanjutan langsung dari Praktikum 1 (Pengenalan Big Data dan Lingkungan Kerja) serta materi kuliah Pertemuan 2 mengenai karakteristik 5V dan lima kategori sumber data. Praktikum ini berfokus pada satu kategori sumber data yang paling sering ditemui dalam praktik, yaitu data transaksional, dengan mensimulasikan data transaksi sebuah platform marketplace fiktif.

Proses membersihkan data mentah menjadi data yang siap dianalisis dikenal sebagai “data preprocessing” , dan merupakan langkah wajib sebelum data dapat disimpan secara terstruktur maupun dijadikan dasar pengambilan keputusan. Praktikum ini secara langsung berkaitan dengan dimensi Veracity pada 5V, yaitu seberapa akurat dan dapat dipercaya suatu data, karena volume data yang besar tidak banyak berguna apabila kualitasnya buruk.

1.2 Tujuan Praktikum

1. Membuat dataset sintetis (synthetic dataset) yang merepresentasikan data transaksi marketplace menggunakan Python.
2. Mengidentifikasi missing value pada dataset dan memilih strategi penanganan yang tepat untuk setiap kolom.
3. Mengidentifikasi baris duplicate dan menghapusnya secara aman.
4. Mengoreksi tipe data yang salah menjadi tipe yang benar, misalnya kolom harga dari teks menjadi numerik.
5. Menstandardisasi format teks dan format tanggal ke satu format baku.
6. Mengekspor hasil akhir ke file transaksi_bersih.csv sebagai deliverable praktikum.

BAB II DASAR TEORI

2.1 Mengapa Data Mentah Perlu Dibersihkan

Data transaksional yang direkam oleh banyak sistem berbeda pada waktu yang berbeda-beda umumnya membawa empat jenis masalah. Pertama, missing value, yaitu kolom kosong karena sistem gagal mencatat atau karena informasinya memang opsional. Kedua, duplicate, yaitu transaksi yang sama tercatat lebih dari sekali, misalnya akibat kegagalan jaringan yang menyebabkan sistem mengirim data dua kali. Ketiga, inconsistent type, yaitu kolom yang seharusnya angka tersimpan sebagai teks karena bercampur simbol seperti "Rp" atau titik ribuan. Keempat, inconsistent format, yaitu kolom yang sama ditulis dalam format berbeda-beda, misalnya tanggal 2026-07-11 dan 11/07/2026 pada baris yang berbeda.

2.2 Kaitan dengan Karakteristik 5V

Keempat masalah pada data mentah tersebut berhubungan langsung dengan dimensi Veracity, yaitu seberapa bisa dipercaya suatu data untuk dijadikan dasar analisis atau pengambilan keputusan. Sebuah dataset dapat saja memiliki Volume besar dan Velocity tinggi, tetapi apabila Veracity-nya rendah, hasil analisis yang dibangun di atasnya berisiko salah. Prinsip ini dikenal secara umum dengan istilah “garbage in, garbage out”.

2.3 Posisi Praktikum dalam Pipeline Big Data

Praktikum ini berada tepat setelah tahap acquisition (data mentah diperoleh dari sumbernya) dan sebelum tahap storage terstruktur yang akan dibahas pada Praktikum 3. Alur posisinya dapat digambarkan sebagai berikut: sumber data melalui proses acquisition menghasilkan data mentah, data mentah melalui proses preprocessing menghasilkan data bersih, dan data bersih pada praktikum berikutnya akan dikonversi ke format Parquet untuk disimpan secara terstruktur.

2.4 Missing Value dan Strategi Penanganannya

Terdapat dua pendekatan umum dalam menangani nilai kosong pada suatu kolom. Pendekatan pertama adalah dropna(), yang membuang seluruh baris apabila nilai pada kolom tertentu kosong. Pendekatan ini tepat digunakan pada kolom yang sifatnya wajib, di mana baris tanpa nilai tersebut tidak dapat lagi dipakai untuk identifikasi maupun analisis. Pendekatan kedua adalah fillna(), yang mengisi nilai kosong dengan nilai pengganti tanpa membuang barisnya, cocok digunakan pada kolom yang sifatnya deskriptif dan tidak berdampak besar terhadap analisis meskipun diisi dengan label netral. Strategi penanganan tidak boleh disamaratakan untuk seluruh kolom, melainkan ditentukan berdasarkan seberapa penting kolom tersebut bagi keabsahan sebuah baris data.

2.5 Duplicate Data

Baris duplicate dapat dideteksi menggunakan `duplicated()`, yang menandai baris apabila seluruh kolomnya identik dengan baris lain, atau dapat pula diperiksa pada kolom kunci tertentu seperti ID transaksi untuk deteksi yang lebih ketat. Setelah terdeteksi, `drop_duplicates()` digunakan untuk membuang salinan kedua dan seterusnya. Urutan eksekusi antara penanganan missing value dan penanganan duplicate perlu diperhatikan, karena membuang baris yang datanya kosong sebelum memeriksa duplicate dapat menyebabkan sebagian pasangan duplicate ikut hilang sebelum sempat terdeteksi secara utuh.

2.6 Koreksi Tipe Data dan Standardisasi Format

Kolom numerik yang tersimpan sebagai teks akibat bercampur simbol mata uang atau pemisah ribuan perlu dibersihkan terlebih dahulu sebelum dikonversi menjadi tipe numerik. Kolom tanggal yang formatnya tercampur, misalnya format ISO (YYYY-MM-DD) bercampur dengan format DD/MM/YYYY, memerlukan penanganan khusus karena penggunaan parameter seperti `dayfirst=True` secara serentak pada seluruh data dapat membalik tanggal yang sebenarnya sudah tidak ambigu, tanpa memunculkan pesan error apa pun. Pendekatan yang lebih aman adalah mencoba beberapa format eksplisit satu per satu untuk setiap nilai.

2.7 Dataset Sintetis dan Faker

Pada praktikum ini digunakan dataset sintetis karena belum tersedia dataset transaksi marketplace otentik yang sesuai untuk latihan. Dataset sintetis dibangkitkan menggunakan pustaka Faker, yang mampu menghasilkan data palsu namun realistis seperti nama orang dan nama kota. Penggunaan random seed yang tetap memastikan hasil pembangkitan data dapat direproduksi secara identik oleh siapa pun yang menjalankan kode yang sama.

BAB III

METODOLOGI PRAKTIKUM

3.1 Waktu dan Lingkungan Praktikum

Praktikum dilaksanakan menggunakan Google Colaboratory (Google Colab) dengan tipe runtime CPU, sesuai arahan modul praktikum. Praktikum ini merupakan Pertemuan Praktikum ke-2 pada Mata Kuliah Praktikum Big Data.

3.2 Perangkat dan Perangkat Lunak

Perangkat dan perangkat lunak yang digunakan selama praktikum dirangkum pada Tabel 3.1 berikut.

Tabel 3.1 Perangkat dan Perangkat Lunak yang Digunakan

Perangkat/Software	Versi/Spesifikasi	Kegunaan
Google Colab	Layanan daring (runtime CPU)	Lingkungan notebook utama untuk menjalankan seluruh kode
Python	3.10 - 3.12	Bahasa pemrograman utama
pandas	≥ 2.0	Pengolahan data tabular utama
NumPy	Terpasang bersama pandas	Pembangkitan angka acak untuk dataset sintetis
Faker	Terbaru (via pip)	Membangkitkan data transaksi sintetis yang realistis

3.3 Dataset

Dataset utama pada praktikum ini adalah data transaksi marketplace sintetis, dibangkitkan menggunakan Python dengan random seed tetap (SEED = 42) agar hasilnya identik dan dapat direproduksi. Dataset mensimulasikan kategori sumber data transaksional yang telah dibahas pada kuliah Pertemuan 2, terdiri atas sepuluh kolom sebagaimana dirangkum pada Tabel 3.2.

Tabel 3.2 Skema Dataset Transaksi Marketplace

Kolom	Tipe	Keterangan
transaction_id	teks	ID unik transaksi, format TRX00001
customer_name	teks	Nama pembeli
product_name	teks	Nama produk
category	teks (kategorikal)	Kategori produk
price	numerik	Harga satuan produk dalam Rupiah
quantity	integer	Jumlah barang dibeli
payment_method	teks (kategorikal)	Metode pembayaran
transaction_date	tanggal	Tanggal transaksi
shipping_city	teks	Kota tujuan pengiriman
rating	numerik (1-5), opsional	Rating dari pembeli, boleh kosong

Dataset ini bersifat sintetis, bukan data transaksi nyata, sebagaimana disyaratkan pada etika penggunaan data di modul praktikum.

3.4 Akses Notebook dan Repository GitHub

Seluruh kode program, output eksekusi, dan artefak praktikum disimpan pada folder Praktikum02 di repository GitHub agar proses pengolahan data dapat diperiksa dan direproduksi kembali.

3.5 Tahapan Praktikum

1. Import library dan inisialisasi random seed.
2. Membuat dataset sintetis sebagai simulasi tahap data acquisition.
3. Mendeteksi dan menangani missing value pada tiap kolom.
4. Mendeteksi dan menghapus baris duplicate.
5. Mengoreksi tipe data dan menstandarisasi format teks serta tanggal.
6. Mengekspor dataset bersih ke file CSV.
7. Mengerjakan latihan, studi kasus, dan pertanyaan evaluasi.

BAB IV

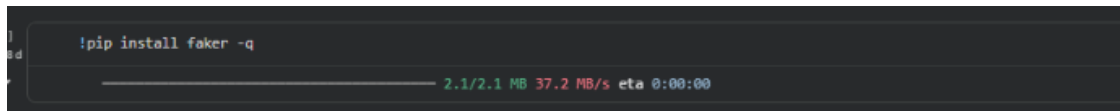
HASIL DAN PEMBAHASAN

4.1 Import Library dan Inisialisasi (K-1)

A. Tujuan

Langkah ini bertujuan menyiapkan seluruh pustaka yang dipakai sepanjang praktikum, yaitu NumPy, pandas, Faker, dan modul random bawaan Python.

B. Kode Program



Gambar 4.1 Instalasi pustaka Faker dan output proses instalasi



Gambar 4.2 Import library dan modul yang digunakan

C. Pembahasan

Empat library yang diimpor memiliki peran berbeda: numpy untuk operasi numerik dan penyedia nilai NaN, pandas sebagai pustaka utama pengolahan data tabular, Faker untuk membangkitkan data palsu yang realistis, dan random untuk memilih nilai acak dari sebuah daftar. Sel ini tidak menghasilkan output karena hanya berisi pernyataan impor.

4.2 Membuat Dataset Sintetis / Simulasi Data Acquisition (K-2)

A. Tujuan

Langkah ini bertujuan mensimulasikan proses acquisition data transaksi, sekaligus dengan sengaja menyisipkan variasi format harga, tanggal, kapitalisasi, missing value, dan baris duplicate, meniru masalah yang biasa muncul pada data mentah di dunia nyata.

B. Kode dan Output Program

```
SEED = 42
np.random.seed(SEED)
random.seed(SEED)
faker = Faker("id_ID")
Faker.seed(SEED)

N = 500
kategori_produk = ["Elektronik", "Fashion", "Kesehatan", "Rumah Tangga", "Olahraga", "Buku"]
metode_bayar = ["Transfer Bank", "E-Wallet", "COD", "Kartu Kredit"]

rows = []
for i in range(1, N + 1):
    trx_id = f"TRX{1:05d}"
    nama_pelanggan = faker.name()
    produk = faker.word().capitalize() + " " + random.choice(["Pro", "Lite", "Max", "Basic", ""])
    kategori = random.choice(kategori_produk)
    harga_dasar = random.choice([15000, 25000, 50000, 75000, 120000, 250000, 500000, 1200000])
    qty = random.randint(1, 5)

    # Variasi format harga: angka polos, ada "Rp", ada desimal ".", ada spasi
    harga_variants = [
        str(harga_dasar),
        f"Rp{harga_dasar:,.0f}".replace(",", "."),
        f"{harga_dasar:,.0f}",
        f"{harga_dasar} ",
    ]
    harga = random.choice(harga_variants)

    # Variasi format tanggal: ISO, DD/MM/YYYY, DD-MM-YYYY
    tgl = fake.date_between(start_date="-50d", end_date="today")
    tgl_variants = [tgl.strftime("%Y-%m-%d"), tgl.strftime("%d/%m/%Y"), tgl.strftime("%d-%m-%Y")]
    tanggal = random.choice(tgl_variants)

    metode = random.choice(metode_bayar)
    if random.random() < 0.3:
        metode = metode.lower()
    if random.random() < 0.2:
        kategori = kategori.upper() + " "

    kota = faker.city()
    rating = random.choice([1, 2, 3, 4, 5, None, None]) # rating optional

    rows.append({
        "transaction_id": trx_id, "customer_name": nama_pelanggan, "product_name": produk.strip(),
        "category": kategori, "price": harga, "quantity": qty, "payment_method": metode,
        "transaction_date": tanggal, "shipping_city": kota, "rating": rating,
    })

df = pd.DataFrame(rows)

# Sertakkan missing value pada beberapa kolom
for col, frac in [("customer_name", 0.02), ("shipping_city", 0.03), ("payment_method", 0.015)]:
    idx = df.sample(frac=frac, random_state=SEED).index
    df.loc[idx, col] = np.nan

# Duplikasi 15 baris (menyalin transaksi yang tercatat dua kali)
dup_rows = df.sample(n=15, random_state=SEED)
df = pd.concat([df, dup_rows], ignore_index=True)
df = df.sample(frac=1, random_state=SEED).reset_index(drop=True)

df.to_csv("transaksi_mentah.csv", index=False)
print("Jumlah baris:", len(df))
```

Gambar 4.3 Kode pembuatan dataset transaksi marketplace sintesis

df.head()

	transaction_id	customer_name	product_name	category	price	quantity	payment_method	transaction_date	shipping_city	rating
0	TRX00305	Ophelia Hartati	Quo Basic	Buku	50000	4	kartu kredit	2028-07-12	Bitar	1.0
1	TRX00500	Cut Maya Wijayanti	Delectus	Elektronik	Rp500.000	1	E-Wallet	2028-07-08	Lubuklinggau	NaN
2	TRX00442	Dr. Ridwan Utama, M.Pd	Animi Max	Elektronik	25000	1	COD	2028-08-24	Probolinggo	2.0
3	TRX00154	Viman Suwama, S.H	Consectetur	Fashion	Rp250.000	1	COD	28/07/2028	Tual	4.0
4	TRX00074	NaN	Eius	Olahraga	250000.0	4	NaN	30/05/2028	NaN	1.0

Langkah berikutnya: [Buat kode dengan df](#) [New interactive sheet](#)

Gambar 4.4 Pratinjau lima baris pertama dataset mentah

C. Pembahasan

Seluruh generator angka acak dikunci menggunakan `np.random.seed()`, `random.seed()`, dan `Faker.seed()` secara bersamaan, karena ketiganya mengendalikan generator yang berbeda dan harus dikunci sekaligus agar proses acak benar-benar dapat direproduksi. Sebanyak 500 baris transaksi dasar dibangkitkan melalui perulangan, masing-masing dengan variasi format harga (empat variasi: angka polos, berprefiks "Rp", berdesimal, dan berspasi), variasi format tanggal (tiga variasi: ISO,

DD/MM/YYYY, DD-MM-YYYY), serta variasi kapitalisasi pada kolom metode pembayaran dan kategori.

Missing value disisipkan menggunakan `df.sample(frac=..., random_state=SEED)` dengan proporsi 2% pada `customer_name`, 3% pada `shipping_city`, dan 1,5% pada `payment_method`. Sebanyak 15 baris diambil dan digabungkan kembali menggunakan `pd.concat()` untuk mensimulasikan transaksi yang tercatat dua kali. Dataset akhir berjumlah 515 baris. Angka ini berasal dari 500 baris dasar ditambah 15 baris duplikasi yang sengaja disisipkan.

4.3 Deteksi dan Penanganan Missing Value (K-3)

A. Tujuan

Langkah ini bertujuan mengidentifikasi jumlah nilai kosong pada setiap kolom, kemudian menentukan strategi penanganan yang berbeda-beda sesuai tingkat kepentingan masing-masing kolom.

B. Kode dan Output Program

```
print(df.isnull().sum())

transaction_id      0
customer_name      20
product_name        0
category            0
price               0
quantity            0
payment_method      16
transaction_date     0
shipping_city       30
rating              166
dtype: int64
```

Gambar 4.5 Hasil pemeriksaan missing value pada setiap kolom

```
df = df.dropna(subset=["customer_name", "payment_method"])
df["shipping_city"] = df["shipping_city"].fillna("Tidak Diketahui")
print("Jumlah baris setelah penanganan missing value:", len(df))

Jumlah baris setelah penanganan missing value: 495
```

Gambar 4.6 Penanganan missing value pada kolom wajib dan shipping_city

C. Pembahasan

Hasil `isnull().sum()` menunjukkan `customer_name` memiliki 20 nilai kosong, `payment_method` 16, `shipping_city` 30, dan `rating` 166. Strategi penanganan ditentukan per kolom, tidak disamaratakan, sebagaimana dirangkum pada Tabel 4.1.

Tabel 4.1 Strategi Penanganan Missing Value per Kolom

Kolom	Strategi	Alasan
customer_name	dropna (buang baris)	Nama pembeli wajib untuk identifikasi transaksi
payment_method	dropna (buang baris)	Metode pembayaran adalah informasi transaksi inti

shipping_city	fillna("Tidak Diketahui")	Baris tetap berguna untuk analisis lain meski kota tidak diketahui
rating	dibiarkan kosong (NaN)	Rating memang opsional, pembeli tidak wajib memberi rating

Setelah `dropna()` diterapkan pada `customer_name` dan `payment_method`, jumlah baris berkurang dari 515 menjadi 495, atau 20 baris terbuang. kedua kolom tersebut dianggap wajib sehingga baris tanpa nilai pada kolom itu tidak dapat dipakai untuk identifikasi transaksi. Kolom `rating` sengaja tidak diisi, karena mengisi nilai kosong dengan angka tebakan berisiko mendistorsi hasil rating rata-rata pada analisis berikutnya.

4.4 Deteksi dan Penanganan Duplicate (K-4)

A. Tujuan

Langkah ini bertujuan mendeteksi baris yang tercatat lebih dari satu kali, kemudian menghapus salinannya sehingga setiap transaksi hanya terwakili oleh satu baris.

B. Kode dan Output Program

```
print("Baris duplicate (semua kolom sama):", df.duplicated().sum())
print("transaction_id duplicate:", df['transaction_id'].duplicated().sum())

df = df.drop_duplicates()
print("Jumlah baris setelah drop_duplicates():", len(df))

Baris duplicate (semua kolom sama): 5
transaction_id duplicate: 5
Jumlah baris setelah drop_duplicates(): 490
```

Gambar 4.7 Deteksi dan penghapusan baris duplicate

C. Pembahasan

Duplicate yang terdeteksi pada langkah ini hanya 5 baris, padahal pada tahap K-2 yang sengaja disisipkan berjumlah 15 baris. hal ini disebabkan oleh urutan eksekusi pipeline. Karena `dropna()` pada K-3 dijalankan lebih dulu sebelum `drop_duplicates()` pada K-4, sebagian pasangan duplicate ikut rusak sebelum sempat terdeteksi secara utuh. Dari 30 baris yang terlibat dalam 15 pasangan duplicate, 20 di antaranya kebetulan memiliki nilai kosong pada `customer_name` atau `payment_method`, sehingga salah satu atau kedua salinannya sudah terbuang lebih dulu oleh `dropna()`. Saat `drop_duplicates()` dijalankan setelahnya, yang masih terdeteksi sebagai duplicate murni tinggal 5 baris. Temuan ini dibahas lebih lanjut pada Bagian 5.2 nomor 2 mengenai urutan eksekusi pipeline yang ideal.

4.5 Koreksi Tipe Data dan Standardisasi Format (K-5)

A. Tujuan

Langkah ini bertujuan menyeragamkan kapitalisasi teks kategorikal, mengonversi kolom harga dari teks bercampur simbol menjadi tipe numerik, menstandarisasi format tanggal ke satu format baku, serta memfinalisasi tipe data seluruh kolom.

B. Kode dan Output Program

a. Standardisasi teks kategorikal (category, payment_method, shipping_city)

```
for col in ["category", "payment_method", "shipping_city"]:
    df[col] = df[col].astype("string").str.strip().str.title()

df["payment_method"] = df["payment_method"].replace({"Cod": "COD"})

print(df["category"].unique())
print(df["payment_method"].unique())

<StringArray>
['Buku', 'Elektronik', 'Fashion', 'Rumah Tangga', 'Kesehatan', 'Olahraga']
Length: 6, dtype: string
<StringArray>
['Kartu Kredit', 'E-Wallet', 'COD', 'Transfer Bank']
Length: 4, dtype: string
```

Gambar 4.8 Standardisasi category, payment_method, dan shipping_city

b. Koreksi tipe data pada kolom price (dari teks bercampur simbol, menjadi numerik)

```
def bersihkan_harga(x):
    if pd.isna(x):
        return np.nan
    x = str(x).strip().replace("Rp", "").replace(".", "").replace(",", ".")
    try:
        return float(x)
    except ValueError:
        return np.nan

df["price"] = df["price"].apply(bersihkan_harga)
df["price"].head()
```

```
price
0    50000.0
1   500000.0
2    25000.0
3   250000.0
5   250000.0

dtype: float64
```

Gambar 4.9 Pembersihan dan konversi kolom price menjadi numerik

c. Standardisasi format tanggal ke YYYY-MM-DD

```
def parse_tanggal(x):
    for fmt in ("%Y-%m-%d", "%d/%m/%Y", "%d-%m-%Y"):
        try:
            return pd.to_datetime(x, format=fmt)
        except ValueError:
            continue
    return pd.NaT

df["transaction_date"] = df["transaction_date"].apply(parse_tanggal).dt.strftime("%Y-%m-%d")
df["transaction_date"].head()
```

	transaction_date
0	2026-07-12
1	2026-07-08
2	2026-08-24
3	2026-07-26
5	2026-09-14

dtype: object

Gambar 4.10 Standardisasi format transaction_date ke YYYY-MM-DD

d. Finalisasi tipe data

```
df["quantity"] = df["quantity"].astype(int)
df["price"] = df["price"].astype(float)
df.dtypes
```

transaction_id	object
customer_name	object
product_name	object
category	string[python]
price	float64
quantity	int64
payment_method	string[python]
transaction_date	object
shipping_city	string[python]
rating	float64

dtype: object

Gambar 4.11 Hasil pemeriksaan tipe data akhir

C. Pembahasan

Standardisasi kapitalisasi menggunakan `.astype("string")` dan bukan `.astype(str)`, karena `.astype(str)` mengubah nilai NaN menjadi teks harfiah "nan" sehingga data kosong tidak lagi terdeteksi sebagai kosong. Setelah standardisasi, category menyisakan 6 nilai unik dan payment_method 4 nilai unik, sesuai jumlah kategori dan metode pembayaran yang didefinisikan pada tahap K-2.

Fungsi `bersihkan_harga()` membersihkan simbol mata uang dan pemisah ribuan sebelum dikonversi ke float, dibungkus dengan try/except ValueError sebagai pengaman agar nilai yang gagal dikonversi menjadi NaN alih-alih menghentikan seluruh proses dengan error.

Kesalahan umum yang harus dihindari pada standardisasi tanggal adalah kode `pd.to_datetime(df["transaction_date"], format="mixed", dayfirst=True)`. kombinasi `format="mixed"` dengan `dayfirst=True` berisiko membalik tanggal yang sebenarnya sudah dalam format ISO yang tidak ambigu, misalnya 2026-07-11 bisa salah terbaca menjadi 2026-11-07. `dayfirst=True` diterapkan ke seluruh nilai tanpa pengecualian, sehingga tanggal ISO yang seharusnya tidak perlu diinterpretasikan ulang ikut terpengaruh. Solusi yang lebih aman, sebagaimana diterapkan pada fungsi `parse_tanggal()`, adalah mencoba beberapa format eksplisit satu per satu untuk setiap nilai.

4.6 Ekspor Dataset Bersih (K-6)

A. Tujuan

Langkah ini bertujuan menyimpan dataset yang sudah melalui seluruh tahap pembersihan ke dalam file CSV sebagai deliverable akhir praktikum.

B. Kode dan Output Program

```
df.to_csv("transaksi_bersih.csv", index=False)
print("Dataset bersih tersimpan:", len(df), "baris")

Dataset bersih tersimpan: 490 baris
```

Gambar 4.12 Ekspor dataset bersih ke transaksi_bersih.csv

C. Pembahasan

File transaksi_bersih.csv berisi 490 baris, dan nama file tersebut dipertahankan persis seperti yang disyaratkan modul karena akan dibaca ulang pada Praktikum 3 dan dikonversi menjadi transaksi_bersih.parquet.

4.7 Analisis Hasil

Ringkasan jumlah baris pada setiap tahap pipeline ditunjukkan pada Tabel 4.2.

Tabel 4.2 Ringkasan Jumlah Baris pada Setiap Tahap Pipeline

Tahap	Sebelum	Sesudah	Terbuang	Keterangan
Data mentah	-	515	-	500 baris dasar + 15 duplikasi
<code>dropna(customer_name, payment_method)</code>	515	495	20	Data wajib kosong
<code>drop_duplicates()</code>	495	490	5	Duplicate murni tersisa
Dataset bersih	-	490	25 total	4,9% dari data mentah

Dari 515 baris data mentah, total 25 baris (sekitar 4,9%) dibuang selama proses pra-pemrosesan. 20 baris dibuang karena kehilangan data wajib pada `customer_name` atau `payment_method`, dan 5 baris dibuang karena terdeteksi sebagai duplicate murni. Kehilangan data selama proses cleaning ini normal dan bukan tanda ada yang salah, justru menjadi bukti bahwa proses cleaning bekerja dengan benar, karena baris yang dibuang memang tidak layak dipakai sebagai dasar analisis.

Kolom rating sengaja tidak diisi (imputed). Seandainya rating kosong diisi dengan angka tebakan, rata-rata rating yang dihitung nanti akan menyimpang dari penilaian pembeli yang sesungguhnya. Rating adalah data subjektif yang tidak wajib diberikan pembeli, sehingga lebih aman dibiarkan kosong dan ditangani secara eksplisit saat dianalisis, misalnya dengan menghitung rata-rata hanya dari transaksi yang memang memiliki rating.

4.8 Studi Kasus Penjadwalan Rekonsiliasi Laporan Penjualan

Studi kasus ini menjawab kejanggalan yang ditemukan pada laporan penjualan bulanan sebuah platform marketplace, di mana total transaksi yang dilaporkan tim IT (515) tidak sama dengan total yang dipakai tim Finance (490).

1. Mengapa angka 515 dan 490 bisa berbeda

Angka 515 adalah jumlah baris pada data mentah sebelum melewati proses apa pun, termasuk baris yang datanya tidak lengkap dan baris yang tercatat dua kali akibat kegagalan sistem. Angka 490 adalah jumlah baris setelah data melewati preprocessing, yaitu setelah baris dengan data wajib kosong dibuang dan baris duplicate dihapus. Selisih 25 baris ini bukan data yang hilang secara tidak sengaja, melainkan baris yang memang tidak memenuhi syarat minimal sebagai transaksi yang valid.

2. Apakah 490 baris “lebih benar” dibanding 515 baris

Dalam konteks Veracity, 490 baris lebih bisa dipercaya untuk dijadikan dasar analisis dibanding 515 baris. Veracity bukan soal jumlah data yang paling banyak, melainkan seberapa akurat dan konsisten data tersebut mencerminkan kejadian yang sebenarnya. Dari 515 baris awal, sebagian adalah catatan ganda dari transaksi yang sama, sehingga jika dihitung apa adanya akan membuat jumlah transaksi maupun total penjualan tercatat lebih besar dari kenyataan. Baris lain yang kehilangan `customer_name` atau `payment_method` juga tidak bisa dipakai untuk identifikasi transaksi secara utuh. Jadi, 490 baris bukan berarti lebih sedikit datanya, tetapi lebih murni mewakili transaksi yang benar-benar valid.

3. Menjelaskan kolom rating yang masih memiliki missing value kepada tim Finance

Kolom rating memang bersifat opsional karena pembeli tidak diwajibkan memberi penilaian setelah transaksi, sehingga cukup banyak baris yang kosong pada kolom ini. Jika nilai kosong tersebut dipaksa diisi dengan angka tebakan, hasil rating rata-rata yang dilaporkan justru akan menyesatkan karena bukan mencerminkan penilaian pembeli yang sesungguhnya. Cara paling jujur menjelaskannya kepada tim Finance adalah dengan menyampaikan bahwa rata-rata rating dihitung hanya dari transaksi yang benar-benar memiliki rating, dan mencantumkan berapa jumlah transaksi yang dipakai sebagai dasar perhitungan tersebut, bukan dari seluruh 490 transaksi.

4.9 Latihan Praktikum

4.9.1 Perbandingan Pipeline pada SEED = 42 dan SEED = 7

Seluruh pipeline K-2 sampai K-4 dibungkus menjadi satu fungsi yang menerima nilai seed sebagai parameter, sehingga hasil dua seed berbeda dapat dibandingkan berdampingan tanpa mengganggu variabel df asli.

1. Ubah SEED menjadi 7 dan jalankan ulang seluruh pipeline. Bandingkan jumlah baris transaksi_mentah.csv dan transaksi_bersih.csv dengan hasil SEED = 42.

```
def jalankan_pipeline(seed, simpan_csv=False):
    np.random.seed(seed)
    random.seed(seed)
    fk = Faker("id_ID")
    Faker.seed(seed)

    rows_ = []
    for i in range(1, N + 1):
        trx_id = f"TRX{i:05d}"
        nama_pelanggan = fk.name()
        produk = fk.word().capitalize() + " " + random.choice(["Pro", "Lite", "Max", "Basic", ""])
        kategori = random.choice(kategori_produk)
        harga_dasar = random.choice([15000, 25000, 50000, 75000, 120000, 250000, 500000, 1200000])
        qty = random.randint(1, 5)
        harga_variants = [str(harga_dasar), f"Rp{harga_dasar:,.0f}".replace(".", ","), f"{harga_dasar}.0", f"{harga_dasar} "]
        harga = random.choice(harga_variants)
        tgl = fk.date_between(start_date="-90d", end_date="today")
        tgl_variants = [tgl.strftime("%Y-%m-%d"), tgl.strftime("%d/%m/%Y"), tgl.strftime("%d-%m-%Y")]
        tanggal = random.choice(tgl_variants)
        metode = random.choice(metode_bayar)
        if random.random() < 0.3:
            metode = metode.lower()
        if random.random() < 0.2:
            kategori = kategori.upper() + " "
        kota = fk.city()
        rating = random.choice([1, 2, 3, 4, 5, None, None])
        rows_.append({"transaction_id": trx_id, "customer_name": nama_pelanggan, "product_name": produk.strip(),
                     "category": kategori, "price": harga, "quantity": qty, "payment_method": metode,
                     "transaction_date": tanggal, "shipping_city": kota, "rating": rating})
```

Gambar 4.13 Kode perbandingan pipeline SEED 42 dan SEED 7 bagian pertama


```

d = pd.DataFrame(rows_)

n_null_rating = 0
for col, frac in [("customer_name", 0.02), ("shipping_city", 0.03), ("payment_method", 0.015)]:
    idx = d.sample(frac=frac, random_state=seed).index
    d.loc[idx, col] = np.nan
    dup = d.sample(n=15, random_state=seed)
    d = pd.concat([d, dup], ignore_index=True)
    d = d.sample(frac=1, random_state=seed).reset_index(drop=True)
    if simpan_csv:
        d.to_csv(f"transaksi_mentah_seed({seed}).csv", index=False)

n_mentah = len(d)
n_rating_kosong = d["rating"].isnull().sum()
n_dup_awal = d.duplicated().sum()

d = d.dropna(subset=["customer_name", "payment_method"])
d["shipping_city"] = d["shipping_city"].fillna("Tidak Diketahui")
n_setelah_dropna = len(d)
n_dup_tersisa = d.duplicated().sum()
d = d.drop_duplicates()
if simpan_csv:
    d.to_csv(f"transaksi_bersih_seed({seed}).csv", index=False)
n_bersih = len(d)

return {
    "mentah": n_mentah,
    "rating_kosong": n_rating_kosong,
    "dup_awal": n_dup_awal,
    "setelah_dropna": n_setelah_dropna,
    "dup_tersisa": n_dup_tersisa,
    "bersih": n_bersih,
}

hasil_42 = jalankan_pipeline(42)
hasil_7 = jalankan_pipeline(7, simpan_csv=True)

perbandingan = pd.DataFrame({
    "SEED = 42": hasil_42,
    "SEED = 7": hasil_7,
})
perbandingan.index = [
    "Baris data mentah",
    "rating kosong",
    "Duplicate sebelum dropna",
    "Baris dataset bersih",
]

```

Gambar 4.14 Kode perbandingan pipeline SEED 42 dan SEED 7 bagian kedua

	"Baris dataset bersih",	
	}	
	print(perbandingan)	
---	SEED = 42	SEED = 7
Baris data mentah	515	515
rating kosong	166	120
Duplicate sebelum dropna	15	15
Baris setelah dropna	495	495
Duplicate tersisa	5	5
Baris dataset bersih	490	490

Gambar 4.15 Output perbandingan pipeline SEED 42 dan SEED 7

Jumlah baris pada setiap tahap sama persis antara SEED 42 dan SEED 7, kecuali jumlah rating kosong yang berbeda (166 berbanding 120). Jumlah baris di tiap tahap ditentukan oleh angka tetap di kode ($N = 500$, $n = 15$ duplikasi, $\text{frac} = 0,02/0,03/0,015$), bukan oleh nilai acak yang dihasilkan, sehingga berapa pun seed-nya jumlahnya akan selalu sama. Kolom rating adalah pengecualian karena diisi lewat `random.choice()` murni tanpa proporsi tetap, sehingga jumlah kosongnya ikut berubah antar seed, namun karena rating tidak dipakai sebagai dasar `dropna()`, perbedaan ini tidak memengaruhi jumlah baris akhir.

4.9.2 Kolom is_valid_price

```
2. Tambahkan kolom is_valid_price bernilai True jika price > 0.

df["is_valid_price"] = df["price"] > 0
print(df["is_valid_price"].value_counts())
print("Jumlah harga tidak valid:", (~df["is_valid_price"]).sum())

is_valid_price
True      490
Name: count, dtype: int64
Jumlah harga tidak valid: 0
```

Gambar 4.16 Pembuatan dan pemeriksaan kolom `is_valid_price`

Seluruh 490 baris pada dataset bersih memiliki harga valid (True), tidak ada satu pun harga bernilai nol atau negatif. Hasil ini membuktikan fungsi `bersihkan_harga()` pada Langkah K-5b berhasil mengonversi seluruh nilai harga menjadi angka yang wajar tanpa ada yang gagal dikonversi.

4.9.3 Jumlah Transaksi per Kategori

```
3. Hitung jumlah transaksi per category menggunakan value_counts().

print(df["category"].value_counts())

category
Olahraga      97
Kesehatan     91
Elektronik    89
Buku          82
Fashion       66
Rumah Tangga  65
Name: count, dtype: Int64
```

Gambar 4.17 Jumlah transaksi pada setiap kategori

Kategori Olahraga memiliki jumlah transaksi terbanyak dengan 97 transaksi, sementara Rumah Tangga tercatat paling sedikit dengan 65 transaksi. Sebaran ini relatif merata karena kategori produk dipilih secara acak seragam menggunakan `random.choice()` pada tahap pembuatan dataset sintetis, dan hasil yang rapi ini turut didukung oleh standardisasi teks yang sudah dilakukan lebih dulu pada Langkah K-5a.

BAB V

TUGAS PRAKTIKUM & PERTANYAAN EVALUASI

5.1 Tugas Praktikum

1. Seluruh pipeline Langkah K-1 sampai K-6 berhasil dijalankan berurutan dari atas ke bawah tanpa error, hasilnya tersimpan pada output tiap sel notebook.
2. Jawaban tertulis untuk ketiga pertanyaan Studi Kasus disajikan pada Bagian 4.8 laporan ini.
3. Hasil ketiga nomor Latihan beserta outputnya disajikan pada Bagian 4.9 laporan ini.
4. File transaksi_bersih.csv hasil akhir (490 baris) dilampirkan bersama notebook sebagai satu paket pengumpulan.

5.2 Pertanyaan Evaluasi

1. Perbedaan dropna() dan fillna(), dan kapan masing-masing dipakai

dropna() membuang seluruh baris yang memiliki nilai kosong pada kolom yang ditentukan, sedangkan fillna() mengisi nilai kosong tersebut dengan nilai pengganti tanpa menghapus barisnya. dropna() sebaiknya dipakai pada kolom yang datanya wajib ada, seperti customer_name dan payment_method pada praktikum ini, karena tanpa nilai tersebut baris menjadi tidak bermakna untuk identifikasi transaksi. fillna() lebih cocok dipakai pada kolom yang tetap berguna meski kosong, seperti shipping_city yang diisi "Tidak Diketahui", karena baris tersebut masih bisa dipakai untuk analisis lain di luar kota pengiriman.

2. Mengapa drop_duplicates() sebaiknya dijalankan pada tahap awal pipeline

Hasil pada praktikum ini justru langsung menunjukkan risikonya. Karena dropna() dijalankan lebih dulu sebelum drop_duplicates(), sebagian pasangan duplicate sudah rusak sebelum sempat terdeteksi. Dari 15 duplicate asli yang sengaja disisipkan, hanya 5 yang masih terdeteksi utuh oleh drop_duplicates(), karena 20 dari 30 baris yang terlibat dalam pasangan duplicate itu kebetulan memiliki nilai kosong pada customer_name atau payment_method dan sudah terlebih dulu dibuang oleh dropna(). Apabila drop_duplicates() dijalankan di awal pipeline, seluruh 15 duplicate akan terdeteksi dan terhapus secara utuh sebelum proses pembersihan lain berjalan, sehingga hasil deteksi duplicate lebih akurat dan tidak bergantung pada urutan langkah pembersihan lainnya.

3. Risiko format="mixed" dengan dayfirst=True pada data tanggal campuran

Ketika kolom tanggal berisi campuran format ISO (YYYY-MM-DD) dan non-ISO (DD/MM/YYYY), penambahan dayfirst=True membuat pandas berasumsi format hari-di-depan berlaku untuk semua nilai, termasuk yang sebenarnya sudah dalam format ISO yang tidak ambigu. Akibatnya tanggal seperti 2026-07-11 yang seharusnya dibaca 11 Juli 2026 bisa salah terbaca menjadi 7 November 2026, karena angka 07 dan 11 dianggap saling tertukar posisi hari dan bulannya. Bahaya utamanya, kesalahan ini tidak memunculkan error sama sekali; tanggalnya tetap berhasil diparsing tetapi nilainya salah, sehingga sulit terdeteksi kecuali dicek manual satu per satu.

4. Mengapa rating tidak diimputasi sedangkan shipping_city diisi "Tidak Diketahui"

Rating adalah data penilaian subjektif dari pembeli, sehingga jika nilai kosongnya diisi angka tebakan, hasil analisis rating rata-rata akan menyimpang dari penilaian yang sesungguhnya terjadi. Berbeda dengan shipping_city yang sifatnya deskriptif dan tidak dipakai untuk perhitungan statistik yang sensitif, sehingga label "Tidak Diketahui" aman dipakai karena tidak mengubah makna analisis lain seperti perhitungan total transaksi atau kategori produk. Prinsipnya, imputasi hanya aman dilakukan selama tidak mendistorsi interpretasi data pada kolom yang bersangkutan.

5. Keterkaitan praktikum ini dengan dimensi Veracity pada 5V

Seluruh proses pada praktikum ini, mulai dari penanganan missing value, penghapusan duplicate, hingga standardisasi format, pada dasarnya adalah usaha meningkatkan Veracity data. Data transaksi mentah yang datang dari berbagai sumber sering kali tidak konsisten dan tidak sepenuhnya bisa dipercaya, meski volumenya besar. Dengan membersihkan data terlebih dahulu, hasil analisis yang dibangun di atasnya menjadi lebih bisa diandalkan, karena Volume data yang besar tidak banyak berguna apabila kualitas atau Veracity-nya rendah.

BAB VI

PENUTUP

6.1 Kesimpulan

1. Pipeline preprocessing pada praktikum ini berhasil mengubah 515 baris data transaksi mentah menjadi 490 baris data bersih melalui penanganan missing value, penghapusan duplicate, koreksi tipe data, dan standardisasi format.
2. Urutan eksekusi tahap pembersihan berpengaruh nyata terhadap hasil deteksi duplicate, terbukti dari jumlah duplicate yang terdeteksi turun dari 15 menjadi hanya 5 karena dropna() dijalankan lebih dulu dibanding drop_duplicates().
3. Dataset bersih transaksi_bersih.csv yang dihasilkan sudah konsisten dari sisi tipe data dan format, sehingga siap dipakai sebagai input pada tahap penyimpanan terstruktur di Praktikum 3.

DAFTAR PUSTAKA

McKinney, W. (2022). Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter (3rd ed.). O'Reilly Media.

pandas Development Team. pandas Documentation. Diakses dari <https://pandas.pydata.org/docs/>

Faker Documentation. Diakses dari <https://faker.readthedocs.io/>

Marz, N., & Warren, J. (2015). Big Data: Principles and Best Practices of Scalable Realtime Data Systems. Manning Publications.

Materi kuliah Pertemuan 2 - Konsep, Sumber, dan Arsitektur Big Data (dokumen internal mata kuliah).

LAMPIRAN

Lampiran 1. Tautan Repository dan Notebook

https://github.com/habibiputrar/BigData26_A_2411531001_Habibi-Putra-Rizqullah-/tree/main/Praktikum02

<https://colab.research.google.com/drive/14UTO9RbhDnCDO3scoE4Ze58gmbMdhNq0?usp=sharing>

Lampiran 2. Tautan Google Drive

File transaksi_bersih.csv (490 baris) dan notebook .ipynb dengan seluruh output tersimpan dilampirkan sebagai satu paket pengumpulan bersama laporan ini tersimpan pada folder Google Drive MyDrive/BigData/Praktikum2 dan pada repository GitHub yang tercantum pada Lampiran 1.

https://drive.google.com/drive/folders/1Vfd4N4ERX38l8ko0TbIWCoF6OeJm_9xr?usp=drive_link